



Wyższa Szkoła Ekonomii i Informatyki w Krakowie

Dokumentacja i serwisowanie
profesjonalnej aplikacji mobilnej lub webowej
Utrzymanie produktu

Rozbudowa systemu



- Wersjonowanie systemu
 - Wersjonowanie semantyczne: <https://semver.org/lang/pl/>
Najważniejsze reguły:
 - System X.Y.Z. W wersji 0.y.z – może się zmieniać wszystko
 - Nie można robić zmian w opublikowanej wersji (tylko nowa wersja)
 - Szybkie zmiany – albo 0.y.z albo osobna gałąź
 - Zmiany nieznaczące (literówki, małe błędy) - Z
 - Zmiany wnoszące nowe funkcjonalności, większe paczki poprawek. Zmiany kompatybilne z aktualną wersją - Y
 - Duże zmiany w produkcie, zmiany które zrywają z kompatybilnością wstecz - X

Rozbudowa systemu



- Procedura podnoszenie wersji aplikacji
 - release notes (zmiany w funkcjonalności, poprawy błędów, zerwanie kompatybilności, zmiany w zależnościach – np. obsługiwana przeglądarka lub używane zewnętrzne oprogramowanie)
 - stały czy zmienny czas/okres? (chrome – 6 tygodni, vscode co miesiąc).
 - osobne podnoszenie po usunięciu błędów krytycznych
- Kompatybilność wsteczna – czy warto i trzeba ją utrzymywać?
 - czy podniesienie wersji produktu przez klienta jest możliwe (koszt, sprzęt, zespół, zewnętrzne integracje)?
 - czy utrzymanie kompatybilności wstecznej blokuje rozwój produktu (rozwój jakościowy, rozwój funkcjonalności)?
 - czy utrzymanie kompatybilności powoduje zwiększenie kosztów utrzymania/rozwoju produktu?

Serwisowanie systemu



- Czy aplikacja backloguje błędy?
- Czy zostały zdefiniowane kategorie błędów i procedury serwisowe do każdej z kategorii (np. czas usunięcia błędu)
- Jak wygląda procedura zgłoszenia i obsługi błędu?
 - Mail/chat/tel/dedyk konsultant
 - Zgłoszenie w aplikacji
 - Autozgłaszanie błędów przez aplikację (+ zewn. narzędzia np. sentry.io)
- Jak wygląda procedura aktualizacji systemu po usunięciu błędu
 - Informacja w release info
 - Informacja w aplikacji
 - Bezpośredni kontakt ze zgłaszającym

Testy wydajnościowe, monitorowanie

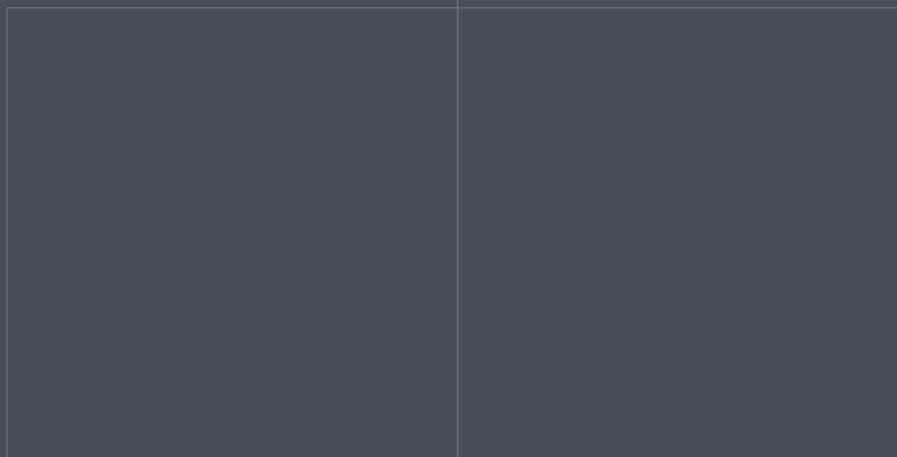


- Badanie oraz monitorowanie wydajności:
 - źródła danych - zapytania sql-owe (czas trwania, najbardziej czasochłonne, najczęstsze)
 - API - czas odpowiedzi i co na niego wpływa – np. obliczenia, sieć, sql etc
 - rps/throughput - request per second
 - interfejsu użytkownika (czasochłonne operacje – np. filtrowanie dużej tabeli, obliczenia, skomplikowana animacja zależna od ilości obiektów na ekranie)
- Określenie granic systemu (dla początkowej konfiguracji wdrożeniowej) – minimalne wymagania aplikacji, maksymalna dopuszczalna ilość użytkowników (bądź inne wskaźniki określające max dopuszczalne zasoby – np. rps)
- Aplikacja webowe – testy na słabszych urządzeniach (urządzenia mobilne) – można częściowo zasymulować w przeglądarce
<https://www.softwaretestinghelp.com/performance-testing-tools-load-testing-tools/>
- Aplikacje mobilne
 - Android Studio -> Android Emulator, Xcode->Simulator,
 - Farmy (np. AWS device farm, Visual Studio App Center, Firebase Test Lab, Perfecto, Bitbar, ...)

Skalowanie



- Wyszczególnienie składowych rozwoju przy skalowaniu aplikacji (np. wzrost liczby użytkowników, złożoności zapytań – zwiększenie stopnia zaawansowania korzystania z aplikacji przez pojedynczego użytkownika – np. korzysta z coraz większych list danych)
 - Wdrożenie cache (front/back/baza)
 - Zachowanie źródeł danych (rozszerzenie, zmiana źródła lub rodzaju źródła danych, optymalizacja zapytań)
 - Zachowanie warstwy logiki aplikacji
 - Zachowanie interfejsów użytkownika
- Skalowanie horyzontalne (wiecej serwerów 😊)/wertykalne (wiecej ramu/cpu) infrastruktury



Wyższa Szkoła Ekonomii
i Informatyki w Krakowie

